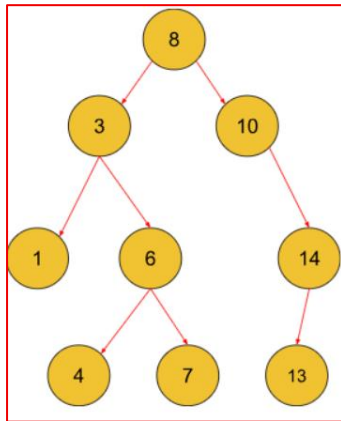


Introduction to Binary Search Trees

Binary Search Trees, popularly known as BST, are the category of Binary Trees used to optimize the operation of searching an element among the Tree Nodes in a Binary Tree.

Let's understand the dynamics of a Binary Search Tree using an illustration below:

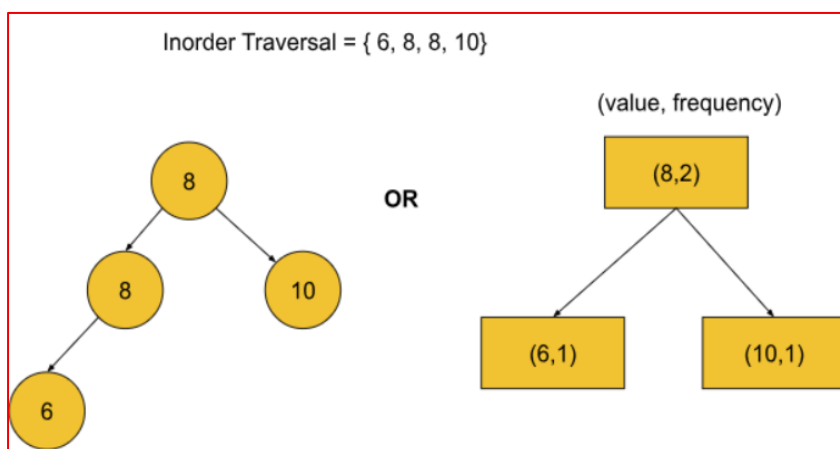


As we can visualize from the above Tree, the right subtree has all the elements greater than the root element and the left subtree has all the elements lesser than the root element. A point to note here is that **every subtree is itself a Binary Search Tree** as it contains a subtree whose root node is lesser than the value of the current node and another subtree whose root node is greater than the value of the current node.

For every given node in a Binary Search Tree if a left or a right child exists then:

Left Child < Node < Right Child

Now, for the general implementation of Binary Search Trees, **duplicate node values are not allowed**. However, in some exceptional cases of Binary Search Tree implementation, we can represent a Binary Search Tree with duplicate node values in the following ways:



After understanding how a Binary Search Tree is represented, let us now understand why is there a need to use a Binary Search Tree instead of a simple Binary Tree.

So, generally in a BST, the maximum height in almost all cases is kept in order of $\log_2(N)$ where N = No. of nodes which is in contrast to the plain Binary Tree whose maximum height can reach the order of N when the tree is **skewed**.

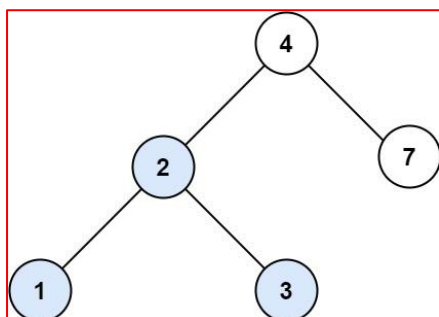
This particularly makes the time of searching for a given node in a Binary Search Tree a lot less than searching in a simple Binary Tree.

For example, if we have to search for an element, we can directly compare it to the root node value of the BST, if the value matches then we have found the element, if say the value is greater than the root node, we say that we now move on to searching that element in the right subtree because the right subtree has all the node values greater than the root value. The search process is terminated for the left subtree as it would only search for those nodes which have values lesser than the root.

Note that, this is very similar to the Binary Search operation which we perform in Arrays to optimize the time taken to search for an element using Linear Search. Hence this category of trees is named **Binary Search Trees**.

Given a Binary Search Tree and a key value return the node in the BST having data equal to 'key' otherwise return null.

Input: root = [4,2,7,1,3], key = 2



Output: [2,1,3]

Optimal:

Algorithm

Step 1: Start with the root node of the Binary Search Tree (BST).

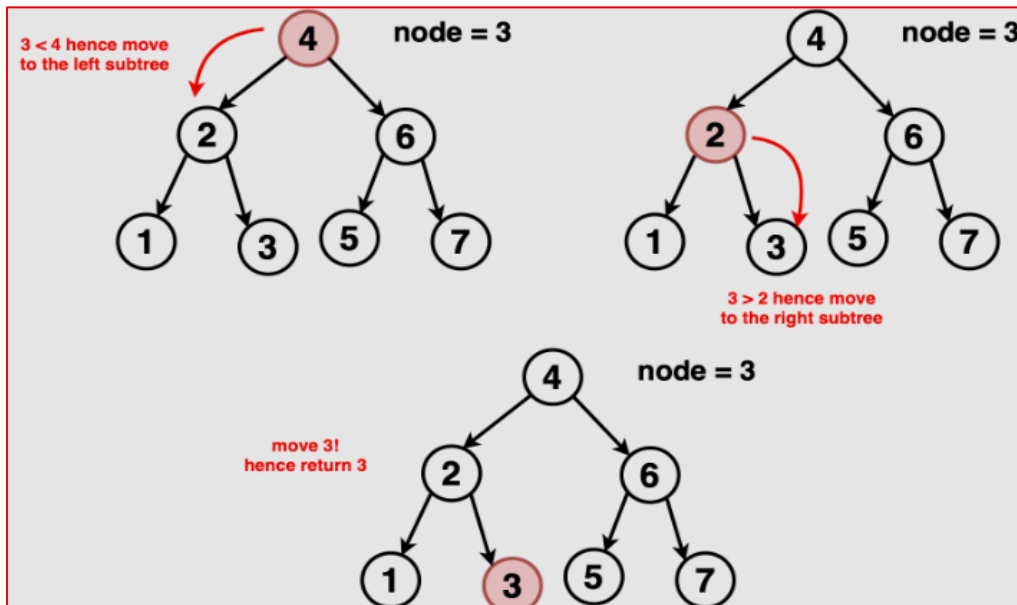
Step 2: Repeat the following steps until the current node becomes null (i.e., the target is not found):

- If the value of the current node is equal to the target value, stop the search (target found).

- If the target value is greater than the current node's value, move to the **right child** of the current node (because all greater values lie in the right subtree).
- Otherwise (if the target value is smaller than the current node's value), move to the **left child** of the current node (because all smaller values lie in the left subtree).

Step 3: If the loop ends and the current node is null, it means the target does not exist in the tree.

Step 4: Return the node containing the target value if found; otherwise, return null.



Code:

```
static Node optimal(Node root,int target){ 1usage
    while (root != null) {
        if (root.data == target) break;
        else if (target > root.data) root = root.right;
        else root = root.left;
    }
    return root;
}
```

Time Complexity:

- **Average / Best Case:** $O(\log N)$ — for balanced BSTs.
- **Worst Case:** $O(N)$ — for skewed (unbalanced) trees.

Space Complexity:

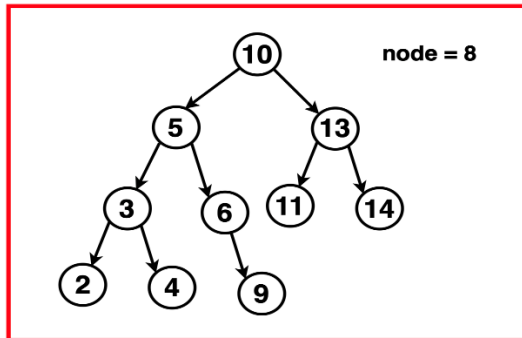
- $O(1)$ — iterative search uses constant extra space.

You are given a root binary search tree and an integer x. Your task is to find the Ceil of x in the tree.

Note: Ceil(x) is a number that is either equal to x or is immediately greater than x.

If Ceil could not be found, return -1.

Input: 10 5 13 3 6 11 14 2 4 -1 9, Key = 8



Output:9

Explanation: In the given BST, the smallest value greater or equal to 8 is 9.

Optimal:

1. Initialize variable res = -1 (this will store the potential answer).
2. Start traversal from the root node.
3. **Repeat until root becomes NULL:**
 - **Case 1:** If root.data >= target
→ The current node could be a potential answer.
→ Store res = root.data.
→ Move to the **left subtree** (root = root.left) because a smaller (and possibly closer) value may exist there that is still $\geq$ target.
 - **Case 2:** If root.data < target
→ The current node and all its left nodes are smaller than target.
→ Move to the **right subtree** (root = root.right).
4. **End of loop:** Return res.

```
static int optimal(Node root, int target){
    int res = -1;

    while(root != null){
        if(root.data >= target){
            res = root.data;
            root = root.left;
        }
        else
            root = root.right;
    }
    return res;
}
```

Time Complexity: $O(h)$

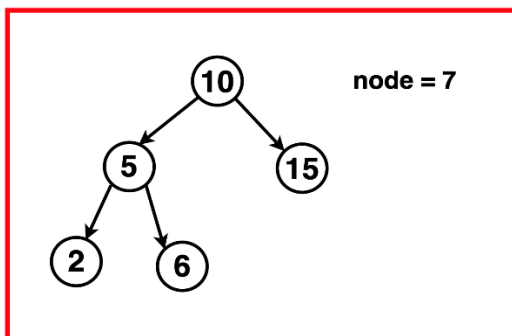
- In the worst case, you may traverse from the root to a leaf $\rightarrow O(h)$, where h = height of the BST.
 - For a balanced BST: $h = \log_2 n \rightarrow O(\log n)$
 - For a skewed BST (like a linked list): $h = n \rightarrow O(n)$

Space Complexity: $O(1)$

You are given a BST(Binary Search Tree) with n number of nodes and value x . your task is to find the greatest value node of the BST which is smaller than or equal to x .

Note: when x is smaller than the smallest node of BST then returns -1.

Input: 10 5 15 2 6 -1 -1, Key = 7



Output: 6

Explanation: In the given BST, the largest value greater or equal to 7 is 6.

Optimal:

1. Initialize $res = -1$ (this will store the potential answer).
2. Start traversal from the root node.

3. Repeat until root becomes NULL:

- **Case 1:** If $\text{root.data} \leq \text{target}$
 - The current node could be a potential answer.
 - Update $\text{res} = \text{root.data}$.
 - Move to the **right subtree** ($\text{root} = \text{root.right}$) Because there might be a larger value still $\leq \text{target}$
- **Case 2:** If $\text{root.data} > \text{target}$
 - Current node and all nodes in its right subtree are greater than target.
 - So, move to the **left subtree** ($\text{root} = \text{root.left}$).

4. End of loop: Return res.

```
public static int floor(Node root, int target) {  
    int res = -1;  
    while(root != null){  
        if(root.data <= target){  
            res = root.data;  
            root = root.right;  
        }  
        else  
            root = root.left;  
    }  
    return res;  
}
```

Time Complexity: $O(h)$

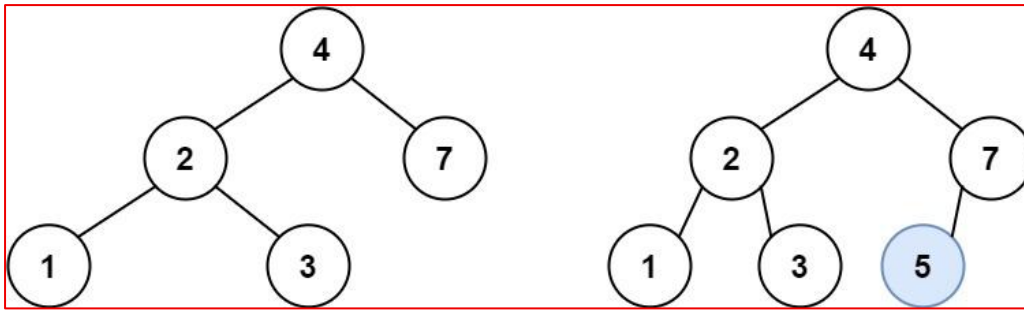
- In the worst case, you may traverse from the root to a leaf $\rightarrow O(h)$, where h = height of the BST.
 - For a balanced BST: $h = \log_2 n \rightarrow O(\log n)$
 - For a skewed BST (like a linked list): $h = n \rightarrow O(n)$

Space Complexity: $O(1)$

You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return *the root node of the BST after the insertion*. It is guaranteed that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them.

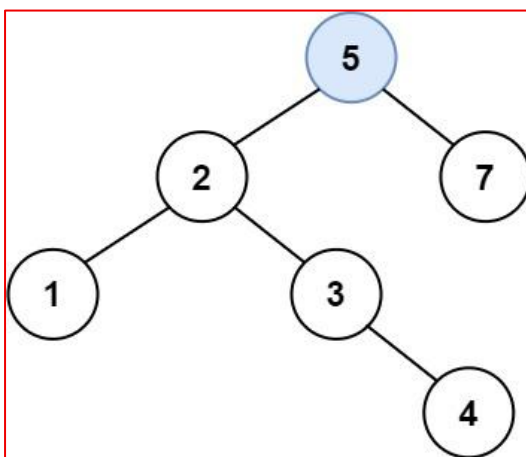
Example 1:



Input: root = [4,2,7,1,3], val = 5

Output: [4,2,7,1,3,5]

Explanation: Another accepted tree is:



Optimal:

Step 1: If root == null,

→ Create a new node with value val.

→ Return this new node as the new root.

Step 2: Store the original root in a variable:

→ curr = root (to keep track of the original root).

Step 3: Start traversing the BST:

→ While root != null, do the following:

Step 4: If val > root.val, then:

- **Step 4.1:** If root.right == null,
→ Create a new node with value val and assign it to root.right.
→ Break out of the loop (insertion completed).
- **Step 4.2:** Else, move to the right subtree → root = root.right.

Step 5: Else ($val < root.val$), then:

- **Step 5.1:** If $root.left == null$,
→ Create a new node with value val and assign it to $root.left$.
→ Break out of the loop.
- **Step 5.2:** Else, move to the left subtree → $root = root.left$.

Step 6: After the loop ends (node inserted),

→ Return $curr$ (the original root of the BST).

```
public TreeNode insertIntoBST(TreeNode root, int val) {
    if(root == null){
        return new TreeNode(val);
    }

    TreeNode curr = root;
    while(root != null){
        if(val > root.val){
            if(root.right == null){
                root.right = new TreeNode(val);
                break;
            }
            else root = root.right;
        }
        else{
            if(root.left == null){
                root.left = new TreeNode(val);
                break;
            }
            else root = root.left;
        }
    }
    return curr;
}
```

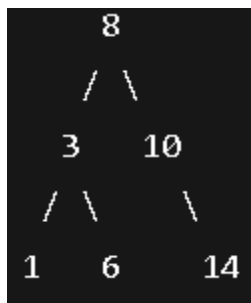
Time Complexity: $O(h)$

- In the worst case, you may traverse from the root to a leaf → $O(h)$, where h = height of the BST.
 - For a balanced BST: $h = \log_2 n \rightarrow O(\log n)$
 - For a skewed BST (like a linked list): $h = n \rightarrow O(n)$

Space Complexity: $O(1)$

Given a BST and an integer. Find the least absolute difference between any node value of the BST and the given integer.

Input:



Suppose the **target** = 7.

We need to find: $\min (| \text{node_value} - \text{target} |)$

for all nodes in the BST:

- $|8 - 7| = 1$
- $|3 - 7| = 4$
- $|10 - 7| = 3$
- $|1 - 7| = 6$
- $|6 - 7| = 1$
- $|14 - 7| = 7$

✓ **Least absolute difference = 1** (from nodes 8 or 6)

Brute-force:

Brute solution can be by exploring each & every node & taking minimum among all. But here the **time-complexity** = $O(n)$.

Optimal:

Step 1: Set $\text{minDiff} = \infty$ (to store the smallest absolute difference found so far)

Step 2: While $\text{root} \neq \text{null}$, do the following:

- Update minimum difference
- If $\text{root.data} == k \rightarrow$ the difference is 0 $\rightarrow$ break the loop
- If $k < \text{root.data} \rightarrow$ move to left subtree ($\text{root} = \text{root.left}$)
- Else $\rightarrow$ move to right subtree ($\text{root} = \text{root.right}$)

Step 3: After traversal, return minDiff .

```

static int optimal(Node root, int k){ 1 usage
    int minDiff = Integer.MAX_VALUE;
    while(root != null){
        minDiff = Math.min(minDiff, Math.abs(root.data - k));
        if(k == root.data)
            break;
        else if(k < root.data)
            root = root.left;
        else
            root = root.right;
    }
    return minDiff;
}

```

Time Complexity: $O(h)$

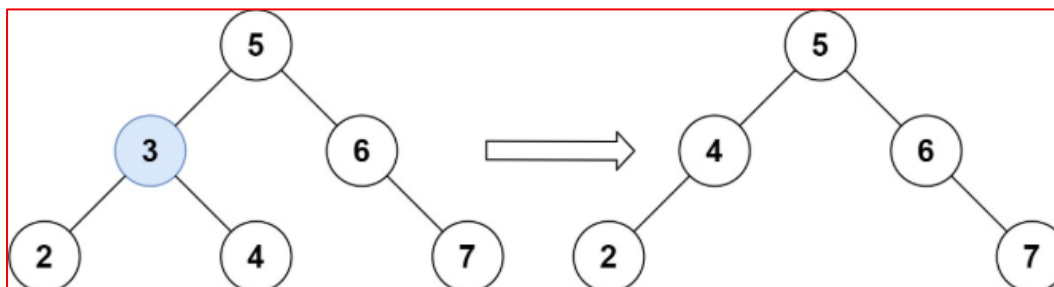
- In the worst case, you may traverse from the root to a leaf $\rightarrow O(h)$, where h = height of the BST.
 - For a balanced BST: $h = \log_2 n \rightarrow O(\log n)$
 - For a skewed BST (like a linked list): $h = n \rightarrow O(n)$

Space Complexity: $O(1)$

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return *the root node reference (possibly updated) of the BST*.

Basically, the deletion can be divided into two stages:

1. Search for a node to remove.
2. If the node is found, delete the node.



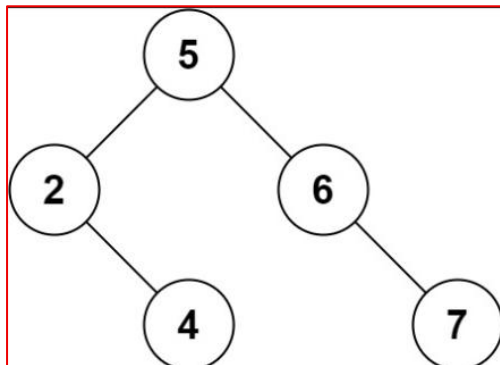
Input: root = [5, 3, 6, 2, 4, null, 7], key = 3

Output: [5, 4, 6, 2, null, null, 7]

Explanation: Given key to delete is 3. So, we find the node with value 3 and delete it.

One valid answer is [5, 4, 6, 2, null, null, 7], shown in the above BST.

Please notice that another valid answer is [5, 2, 6, null, 4, null, 7] and it's also accepted.



Optimal:

1.Base Case:

- If `root == null`, return null (empty tree or target not found).

2.If the target node is the root itself:

- If `root.data == target`, call the `helper()` function on root to delete it.
- Return the result from `helper()` as the new root.

3.Initialize a pointer **temp = root** to store the reference to the original root (used for returning later).

4.Traverse the BST to locate the node to be deleted:

- While `root != null`:
 - If `target < root.data`:
 - Check if `root.left` exists and `root.left.data == target`.
 - If yes, perform deletion using `helper(root.left)` and assign the result to `root.left`.
 - Break the loop.
 - Else, move `root = root.left`.
 - Else (`target > root.data`):
 - Check if `root.right` exists and `root.right.data == target`.
 - If yes, perform deletion using `helper(root.right)` and assign the result to `root.right`.

- Break the loop.
- Else, move root = root.right.

5. Return temp (original root) as the updated root of the tree.

```
static TreeNode helper(TreeNode root){
    if (root.left == null) return root.right;
    else if (root.right == null) return root.left;

    TreeNode rightChild = root.right;
    TreeNode leftRight = findLeftRight(root.left);
    leftRight.right = rightChild;
    return root.left;
}
static TreeNode findLeftRight(TreeNode root){
    while (root.right != null) root = root.right;
    return root;
}
public TreeNode deleteNode(TreeNode root, int target) {
    if (root == null) return null;
    if (root.val == target)
        return helper(root);

    TreeNode temp = root;
    while (root != null){
        if (root.val > target){
            if (root.left != null && root.left.val == target){
                root.left = helper(root.left);
                break;
            }
            else root = root.left;
        }
        else{
            if (root.right != null && root.right.val == target){
                root.right = helper(root.right);
                break;
            }
            else root = root.right;
        }
    }
    return temp;
}
```

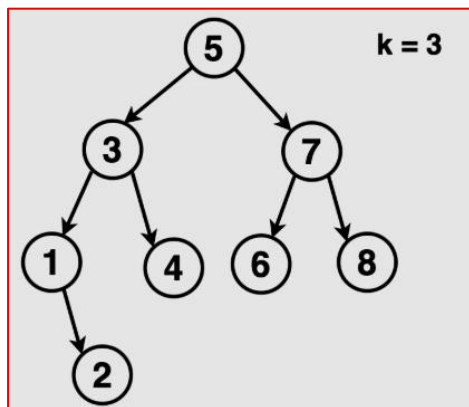
Time Complexity: $O(h)$ → where h is the height of the BST

- For a balanced BST → $O(\log n)$
- For an unbalanced BST → $O(n)$

Space Complexity: $O(1)$

Given the root of a binary search tree, and an integer k, return the k^{th} smallest value (1-indexed) of all the values of the nodes in the tree.

Input:



Output: 3rd smallest: 3

Explanation: All the elements of the BST in the sorted order would be: [1, 2, 3, 4, 5, 6, 7, 8]. From this array it is evident that the index of the Kth smallest element would be K-1. Hence 3rd smallest = 3.

Brute-Force:

A brute force approach would be to traverse the BST using an in-order traversal which will get us an array containing all node values in the ascending sorted order. Following this traversal, we retrieve the Kth smallest element by accessing the element at index k-1 in the array considering indices start from 0.

Algorithm:

Step 1: Initialize an empty list

- Create an empty `ArrayList<Node>` temp to store the result of the inorder traversal.

Step 2: Perform Inorder Traversal

- Call the recursive function `inorder(root, temp)`.

Step 7: Check if kth element exists

- If $k > \text{temp.size}()$, return -1 (kth smallest element doesn't exist).

Step 8: Return the kth smallest element

- Return `temp.get(k-1).data` (0-based indexing).

```

static void inorder(Node root, ArrayList<Node> temp){
    if (root == null) return;
    inorder(root.left, temp);
    temp.add(root);
    inorder(root.right, temp);
}

static int bruteForce(Node root, int k){ 1 usage
    ArrayList<Node> temp = new ArrayList<>();
    inorder(root, temp);

    if (k > temp.size()) return -1;
    return temp.get(k-1).data;
}

```

Time Complexity: $O(N)$ Each node is visited exactly once (where N = total number of nodes)

Space Complexity: $O(N+H)$

- **Auxiliary space for recursion: $O(H)$**
(where H is the height of the tree)
- **Extra space for list: $O(N)$**
(to store all nodes in sorted order)

Optimal:

A more efficient approach for finding the K -th smallest elements in a (BST) without using extra space would involve an optimised traversal technique directly targeting the K -th elements without storing all elements in an array. We use traversal methods (inorder) to find the K th smallest in the given BST. We maintain a counter variable to track the number of visited nodes, stopping when the K th element is found in each traversal.

Algorithm:

Step 1: Create an empty list 'res' to store the result once the k th element is found.

Step 2: Use an integer array count[1] to keep track of the number of nodes visited in inorder traversal.

Step 3: Recursive helper function (helper):

- Base case: If $root == null$ or ans is already non-empty, return.
(Stops recursion early once the k th element is found)
- Recursively traverse the left subtree: $helper(root.left, k, count, ans)$
- After left traversal, increment $count[0]$ by 1.

- If `count[0] == k`, the current node is the kth smallest:
→ Add it to `ans` and return immediately.
- Recursively traverse the right subtree: `helper(root.right, k, count, ans)`

Step 4: Call the helper function with the root node:

Step 5: Return the result:

- If `res` is empty, return -1 (`k` is larger than total nodes).
- Otherwise, return `res.get(0).data` (the kth smallest element).

```
static void helper(Node root, int k, int[] count, ArrayList<Node> ans){
    if (root == null || !ans.isEmpty()) return;
    helper(root.left, k, count, ans);
    if (!ans.isEmpty()) return;
    count[0] += 1;
    if (count[0] == k) {
        ans.add(root);
        return;
    }
    helper(root.right, k, count, ans);
}

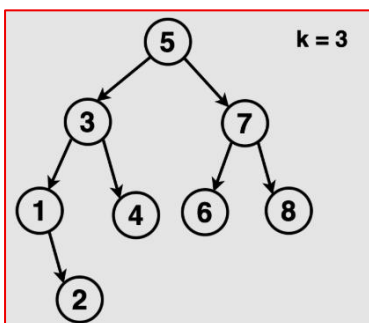
static int optimal(Node root, int k){ 1 usage
    ArrayList<Node> res = new ArrayList<>();
    helper(root, k, new int[1], res);
    if (res.isEmpty()) return -1;
    return res.getFirst().data;
}
```

Time Complexity: $O(k)$ Stops recursion once kth element is found → traverses at most `k` nodes.

Space Complexity: $O(H)$ Auxiliary space for recursion.

Given the root of a binary search tree, and an integer `k`, return the k^{th} smallest value (1-indexed) of all the values of the nodes in the tree.

Input:



Output: 3rd largest: 3

Explanation: All the elements of the BST in the sorted order would be: [1, 2, 3, 4, 5, 6, 7, 8]. From this array it is evident that the index of the Kth smallest element would be (List size - k). Hence 3rd smallest = 6.

Brute-force:

Same as the finding kth smallest in the list by just returning element at (List size - k) index in place of returning (k-1) index element.

```
static void inorder(Node root, ArrayList<Node> temp){
    if (root == null) return;
    inorder(root.left,temp);
    temp.add(root);
    inorder(root.right,temp);
}

static int bruteForce(Node root, int k){ 1 usage
    ArrayList<Node> temp = new ArrayList<>();
    inorder(root,temp);

    if (k > temp.size()) return -1;
    return temp.get(temp.size()-k).data;
}
```

Time Complexity: $O(N)$ Each node is visited exactly once (where N = total number of nodes)

Space Complexity: $O(N+H)$

- **Auxiliary space for recursion: $O(H)$**
(where H is the height of the tree)
- **Extra space for list: $O(N)$**
(to store all nodes in sorted order)

Optimal:

Same as the finding kth smallest in the list by just doing reverse-inorder traversal(right root left).


```

static void helper(Node root, int k, int[] count, ArrayList<Node> ans){
    if (root == null || !ans.isEmpty()) return;
    helper(root.right,k,count,ans);
    if (!ans.isEmpty()) return;
    count[0] += 1;
    if (count[0] == k) {
        ans.add(root);
        return;
    }
    helper(root.left,k,count,ans);
}

static int optimal(Node root, int k){ 1 usage
    ArrayList<Node> res = new ArrayList<>();
    helper(root,k,new int[1],res);
    if (res.isEmpty()) return -1;
    return res.getFirst().data;
}

```

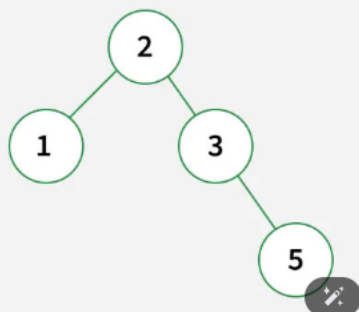
Time Complexity: $O(k)$ Stops recursion once kth element is found $\rightarrow$ traverses at most k nodes.

Space Complexity: $O(H)$ Auxiliary space for recursion.

Given the root of a binary tree. Check whether it is a BST or not. A BST is defined as follows:

- The left subtree of a node contains only nodes with data less than the node's data.
- The right subtree of a node contains only nodes with data greater than the node's data.
- Both the left and right subtrees must also be binary search trees.

Input: root = [2, 1, 3, N, N, N, 5]



Output: true

Explanation: The left subtree of every node contains smaller data and right subtree of every node contains greater data. Hence, the tree is a BST.

Optimal:

Step 1: Start with the entire valid range

- Begin at the root of the tree.
- Allow the root to take **any integer value**, so the initial allowed range is **minimum = $-\infty$, maximum = $+\infty$** .

Intuition:

Since the root is the first node, it can have any possible value.

Step 2: If the current node is empty, accept it

- When you reach a null (empty) child, consider it valid.

Intuition:

An empty subtree cannot violate BST properties.

Step 3: Check whether the node's value is within the allowed range

- For the current node, confirm that:
 - Its value is strictly greater than the minimum allowed value.
 - Its value is strictly less than the maximum allowed value.

Intuition:

This ensures every node conforms not only to its parent but also to all ancestors.

Step 4: Recursively validate the left subtree

- Move to the left child.
- The left child must be less than the current node.
- So:
 - Keep the same minimum allowed value.
 - Update the maximum allowed value to the current node's value.

Intuition:

All values in the left subtree must be smaller than the current node.

Step 5: Recursively validate the right subtree

- Move to the right child.
- The right child must be greater than the current node.
- So:
 - Keep the same maximum allowed value.
 - Update the minimum allowed value to the current node's value.

Intuition:

All values in the right subtree must be greater than the current node.

Step 6: Combine results of left and right subtrees

- If **both** the left subtree and the right subtree are valid, then the entire tree up to this node is valid.
- If **either one fails**, then the whole tree is not a BST.

Intuition: A BST is valid only when every subtree follows proper ordering rules.

```
static boolean helper(Node root,int minVal, int maxVal){ 3 usages
    if (root == null) return true;
    if (root.data<=minVal || root.data>=maxVal) return false;

    boolean l = helper(root.left,minVal,root.data);
    boolean r = helper(root.right,root.data,maxVal);

    return l && r;
}
static boolean optimal(Node root){ 1 usage
    return helper(root, Integer.MIN_VALUE,Integer.MAX_VALUE);
}
```

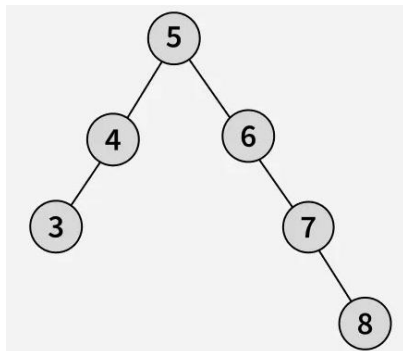
→ **TC = O(N)** Visit each node once, Where N = number of nodes in the tree.

→ **SC = O(H)** Auxiliary space for recursion. Where H = height of tree.

Given the root of a Binary Search Tree (with all values unique) and two nodes n1 and n2 (n1 != n2). You may assume that both nodes exist in the tree. Find the Lowest Common Ancestor (LCA) of the given two nodes in the BST.

Note: LCA between two nodes n_1 and n_2 is defined as the deepest node that has both n_1 and n_2 as descendants (where we allow a node to be a descendant of itself).

Input: root = [5, 4, 6, 3, N, N, 7, N, N, 8], $n_1 \rightarrow \text{data} = 7$, $n_2 \rightarrow \text{data} = 8$



Output: 7

Explanation: 7 is the lowest node that has both 7 and 8 as descendants.

Brute-force: If you want the LCA of two nodes, you can:

- Build the **path** from root to t_1
- Build the **path** from root to t_2
- The LCA is the **last common value** present in both paths

Implementation:

Step 1: Create two empty lists

- One for storing the path from root to **t_1**
 - Another for storing the path from root to **t_2**
-

Step 2: Build the path to t_1

Use the helper function to:

- Start at the root
 - Add every node's value you pass to the list
 - Move left or right based on whether the target is smaller or larger
 - Stop when the target is found
-

Step 3: Build the path to t_2

Repeat the same process as above, but for target **t2**.

Step 4: Find the last common element

Now compare:

- Every element from path1
- With every element from path2

Whenever a value is present in both lists, store it as the current answer.

Since the paths are stored from **root** → **downwards**,
the **last** value that matches in both lists is the **Lowest Common Ancestor**.

Step 5: Return the final common value

```
static void helper(Node root, int target, List<Integer> path){
    while (root!=null){
        path.add(root.data);
        if (root.data==target) break;
        else if(target < root.data) root=root.left;
        else root=root.right;
    }
}

static int bruteForce(Node root,int t1, int t2){ 1 usage
    List<Integer> path1 = new ArrayList<>();
    List<Integer> path2 = new ArrayList<>();

    helper(root,t1,path1);
    helper(root,t2,path2);
    int res = root.data;
    for (int e1: path1){
        for (int e2: path2){
            if (e1==e2) res = e1;
        }
    }
    return res;
}
```

T.C: $2*O(H) + O(H*H)$

$2*O(H)$: for helper function.

$O(H*H)$: for comparing both the paths

S.C: $2*O(H)$ for storing both the paths

Optimal:

Step 1: Start at the root

Step 2: Compare both target values with the current node

Step 3: If both targets are larger than current node.

Move to the **right child**.

Intuition:

Both t1 and t2 lie in the right subtree, so the LCA must also lie there.

Step 4: If both targets are smaller than current node

Move to the **left child**.

Intuition:

Both t1 and t2 lie in the left subtree, so the LCA must be somewhere in the left branch.

Step 5: If they fall on different sides (or match the root)

This is the moment LCA is found.

Why?

- One value is on the left, the other is on the right
or
- One of them *is* the current node

In either case, this node is the **lowest** common point where the two paths diverge.

You stop here and consider this node as the answer.

Step 6: Return the result.

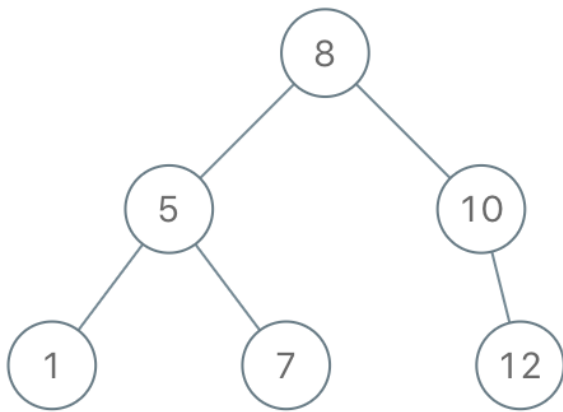
```
static int optimal(Node root, int t1, int t2){ 1 usage
    int ans = root.data;
    while (root != null){
        if (t1 > root.data && t2 > root.data) root = root.right;
        else if (t1 < root.data && t2 < root.data) root = root.left;
        else {
            ans = root.data;
            break;
        }
    }
    return ans;
}
```

Time = $O(H)$ as we are traversing the tree once only. 'H' height of tree.

Space = $O(1)$

Given an array of integers preorder, which represents the preorder traversal of a BST (i.e., binary search tree), construct the tree and return *its root*.

Input: preorder = [8,5,1,7,10,12]



Output: [8,5,10,1,7,null,12]

Brute-force:

Preorder traversal represents nodes in the order:

root → left-subtree → right-subtree

If you insert elements of a preorder array one by one into an empty BST using normal BST insertion rules, you will end up reconstructing the **same original BST**.

Why?

- The **first element** of preorder is always the **root**.
- All remaining elements will fall either:
 - into the left subtree (when less than root), or
 - into the right subtree (when greater than root)
- In BST insertion, each number naturally finds the correct position based on comparisons.

So, this brute-force solution simply **builds the BST exactly as it would grow naturally** if these elements were inserted one by one.

Implementation:

Step 1: Create the root

- The first value of the preorder array becomes the root node.
 - This is because preorder always starts with the root.
-

Step 2: Loop through the remaining values

For each remaining number:

- Treat it as a new node that must be inserted into the BST.
-

Step 3: Start searching from the root

- Begin at the root node.
 - Compare the value to be inserted with the current node's value.
-

Step 4: Move left or right based on BST rules

- If the new value is smaller, go to the **left child**.
 - If the new value is larger, go to the **right child**.
-

Step 5: Find the correct empty position

Keep moving left or right until:

- You find a null (empty) spot in the correct direction
- Insert the new node there

This insertion accurately reflects where the node would belong in the original BST.

Step 6: Repeat for every element

Continue the process for all elements in the preorder array.

Once all elements are inserted, you have reconstructed the BST.

```
static Node bruteforce(int[] arr){ no usages
    Node root = new Node(arr[0]);
    for (int i=1;i< arr.length;i++){
        Node curr = root;
        while (true){
            if (arr[i]<curr.data) {
                if (curr.left == null) {
                    curr.left = new Node(arr[i]);
                    break;
                }
                else curr = curr.left;
            }
            else if (arr[i]>curr.data) {
                if (curr.right == null) {
                    curr.right = new Node(arr[i]);
                    break;
                }
                else curr = curr.right;
            }
        }
    }
    return root;
}
```

T.C: $O(H) + O(N)$ H=height of tree & N=no.of elements in array.

$O(H)$: To insert one value, you start at the root and walk down the tree until you find the correct position.

$O(N)$: You repeat the insertion process for **each of the N elements**

Better-Approach:

By sorting given pre-order & store it in some other array(in-order) as we know in in-order we always get elements in sorted order. So, by using both pre-order & in-order traversal we can create a bst like we did in Binary tree.

T.C: $O(\log n) + O(n)$ where 'n' is the size of array.

$\log n$: for sorting,

n: for creating bst.

S.C: $O(n)$ for storing in-order traversal.

Optimal-approach:

So, while rebuilding:

- The first unused value is always the next node to place.
- But you must ensure it follows BST rules.
- The **max-bound** ensures a value belongs to the current subtree.

Example intuition:

- When building the left subtree of a node X, any value must be $< X$
- When building the right subtree, any value must be $< \text{parent's max limit}$

Implementation:

Step 1: Start with the first element of preorder

- Create a pointer (index = 0)
 - This pointer tracks which preorder value is next to use.
 - The first value becomes the root.
-

Step 2: Begin recursive construction using bounds

Call the helper with:

- The array
 - A maximum allowed value for this subtree
 - The index pointer
-

Step 3: Base condition

If:

- You have used all elements **or**
- The next element is **greater than the allowed max**

→ This position cannot accept that value.

→ Return null (no node here).

Step 4: Create a node

Take the value at current index and create a new node.

Increase the pointer so that next call uses the next element.

Step 5: Build the left subtree

Call helper for the left child with:

- A new max-bound = current node's value
(left subtree nodes must be smaller)

Left subtree consumes some number of preorder elements.

Step 6: Build the right subtree

Call helper for the right child with:

- The old max-bound
(right subtree can accept values up to parent's limit)

Right subtree consumes the next unused preorder elements.

Step 7: Return node

Once both subtrees are constructed, return the node upward.

```
static Node helper(int[] arr, int max, int[] index) { 3 usages
    if (index[0] == arr.length || arr[index[0]] > max) return null;
    Node curr = new Node(arr[index[0]]);
    index[0] += 1;
    curr.left = helper(arr, curr.data, index);
    curr.right = helper(arr, max, index);
    return curr;
}
static Node optimal(int[] arr){ 1 usage
    return helper(arr, Integer.MAX_VALUE, new int[] {0});
}
```

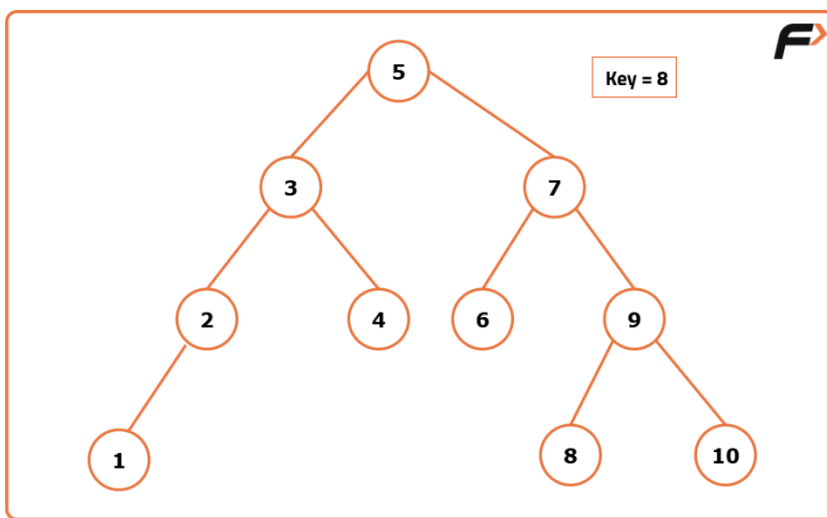
→ **Time = $O(N)$** Each value in preorder is processed exactly once:

Space = $O(H)$ recursion stack space.

You are given the root of a BST and an integer key. You need to find the inorder predecessor and successor of the given key. If either predecessor or successor is not found, then return -1.

Note: In an inorder traversal the number just smaller than the target is the predecessor and the number just greater than the target is the successor.

Input: Binary Search Tree: 5 3 7 2 4 6 9 1 -1 -1 -1 -1 -1 8 10, Key = 8



Output: Inorder Predecessor: 7, Inorder Successor: 9

Brute-force:

First, we will do the in-order traversal & store its value inside the list & after that will do the binary search to find the index of 'k' once we get the index we can return successor as 'index+1' element & predecessor as 'index-1' element.

Time Complexity: $O(N + \log N)$ where N is the number of nodes of the binary search tree. $O(N)$ to traverse all nodes and store them in an in-order array and $O(\log N)$ for the binary search.

Space Complexity: $O(N)$ + recursion stack space as an array of size N is used to store the in-order traversal of the binary search tree.

Better-Solution: by doing in-order traversal during this whenever we found 'any value > k' & successor == -1 replace 'successor = value' & return 'successor' at the end.

Time Complexity: $O(N)$ where N is the number of nodes in the binary search tree.

Space Complexity: $O(1)$ + recursion stack space

Optimal:

- Initialize a variable 'successor = -1'.
- Traverse the Binary Search Tree (BST) based on the key value.
- If the current node value is smaller than the key, move to its right child.
- If the current node value is greater than the key, update 'successor' with current node value, then move to the left child.
- Continue this process until the traversal reaches a null node.
- After that will return 'successor'.

Time Complexity: $O(H)$ where H is the height of the binary search tree as we are traversing along the height of the tree.

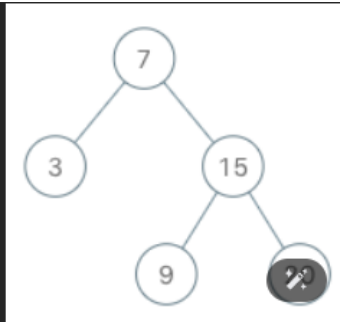
Space Complexity: $O(1)$

Implement the BSTIterator class that represents an iterator over the [in-order traversal](#) of a binary search tree (BST):

- **BSTIterator(TreeNode root)** Initializes an object of the BSTIterator class. The root of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- **boolean hasNext()** Returns true if there exists a number in the traversal to the right of the pointer, otherwise returns false.
- **int next()** Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to next() will return the smallest element in the BST.

You may assume that next() calls will always be valid. That is, there will be at least a next number in the in-order traversal when next() is called.



Input

```
["BSTIterator", "next", "next", "hasNext", "next", "hasNext", "next",
"hasNext", "next", "hasNext"]
[[[7, 3, 15, null, null, 9, 20]], [], [], [], [], [], [], [], [], []]
```

Output

```
[null, 3, 7, true, 9, true, 15, true, 20, false]
```

Explanation

```
BSTIterator bSTIterator = new BSTIterator([7, 3, 15, null, null, 9, 20]);
bSTIterator.next();      // return 3
bSTIterator.next();      // return 7
bSTIterator.hasNext();   // return True
bSTIterator.next();      // return 9
bSTIterator.hasNext();   // return True
bSTIterator.next();      // return 15
bSTIterator.hasNext();   // return True
bSTIterator.next();      // return 20
bSTIterator.hasNext();   // return False
```

Brute-force: By storing all the elements in the ArrayList by doing in-order traversal. Also we initially set a variable 'index = 1' & whenever next() will be called will return list[index++] & whenever hasNext() will be called will return true if list is not empty otherwise, false.

tc= O(n) for in-order traversal using recursion.

sc = O(n) for storing value in list+ recursion stack space.

Optimal: By using stack & initially will store all the elements from root till left-most node & whenever next() will be called will return top() element by pushing node right & till left-most values in the stack. whenever hasNext() will be called will return true if stack is not empty otherwise false.

```

class BSTIterator {

    private Stack<TreeNode> st = new Stack<>();

    public BSTIterator(TreeNode root) {
        addAll(root);
    }

    public int next() {
        TreeNode curr = st.pop();
        addAll(curr.right);
        return curr.val;
    }

    public boolean hasNext() {
        return !st.isEmpty();
    }

    public void addAll(TreeNode root){
        while(root!=null){
            st.push(root);
            root=root.left;
        }
    }
}

```

tc=O(n) in worst we might have to explore all the nodes.

sc = O(H) at-most stack will store elements equal to the height of tree.

Given the root of a binary search tree and an integer k, return true if there exist two elements in the BST such that their sum is equal to k, or false otherwise.

Brute-force:

First we will do in-order traversal & store the traversal in arraylist after that will use two pointer technique to check whether sum of any element in the list is equal to 'k' or not by pointer 'i' at the start & 'j' at the end of list.

```

static void helper(Node root, ArrayList<Integer> inOrder){
    if (root == null) return;
    helper(root.left,inOrder);
    inOrder.add(root.data);
    helper(root.right,inOrder);
}

static Boolean bruteForce(Node root,int k){ 1 usage
    ArrayList<Integer> inOrder = new ArrayList<>();
    helper(root,inOrder);

    int i=0, j=inOrder.size()-1;
    while (i<j){
        if (inOrder.get(i)+inOrder.get(j) == k) return true;
        else if (inOrder.get(i)+inOrder.get(j) > k) j=j-1;
        else i=i+1;
    }
    return false;
}

```

T.C: $O(2n)$ $O(n)$ for in-order traversal & another $O(n)$ for two-pointer approach.

S.C: $O(n)$ + recursion stack space

Optimal:

We know Inorder traversal of a BST gives a **sorted array**. But instead of storing all elements (which takes **$O(N)$** extra space), we simulate the same behavior using **two iterators**:

- 1** Forward Iterator (next()) → Smallest to Largest : It uses a stack (st1) that stores the path to the next largest number.
- 2** Backward Iterator (before()) → Largest to Smallest: It uses another stack (st2) that stores the path to the next smallest number.

Implementation:

✓ Step 1: Create Two Stacks

- st1 → for normal inorder (small → large)
- st2 → for reverse inorder (large → small)

✓ Step 2: Initialize both stacks

During object creation, we push:

- All left nodes into st1
- All right nodes into st2

This positions:

- next() at the smallest node
- before() at the largest node

✓ Step 3: Implement next()

- Pop the top node of st1
- That gives current **smallest**
- Push its right subtree's left chain (just like inorder)

✓ Step 4: Implement before()

- Pop the top node of st2
- That gives current **largest**
- Push its left subtree's right chain (reverse inorder)

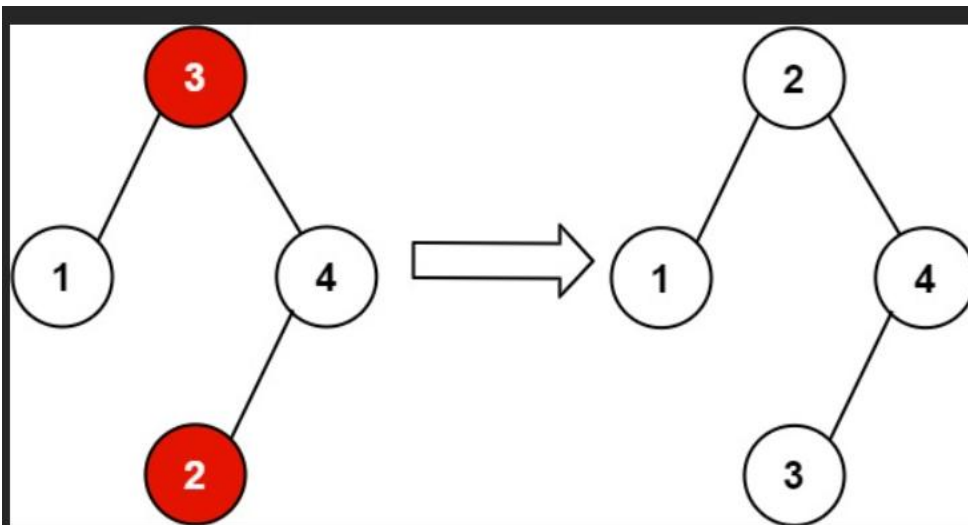
✓ Step 5: Two-pointer logic

- Let i = next() (min value)
- Let j = before() (max value)
- While i < j:
 - If i + j == k → TRUE
 - If i + j < k → move forward → i = next()
 - If i + j > k → move backward → j = before()

Time = O(N) Because each node is processed at most once across all operations: push → pop → maybe pushed again by iterator subtree traversal.

Space: 2*O(H) at-most both stack will store elements = height of tree.

You are given the root of a binary search tree (BST), where the values of exactly two nodes of the tree were swapped by mistake. Recover the tree without changing its structure.



Input: root = [3,1,4,null,null,2]

Output: [2,1,4,null,null,3]

Explanation: 2 cannot be in the right subtree of 3 because $2 < 3$. Swapping 2 and 3 makes the BST valid.

Brute-force:

First we will do the in-order traversal & store it in the array-list after that we will sort the arraylist in ascending order after that we will start iterating the given tree in Inorder fashion by pointing a flag variable on the starting element of arraylist(represents index) & whenever we find any element which is not in the correct order will simply replace value of that node with the value where 'flag' is pointing in the list.

Time: $2O(n) + O(n \log n)$ $2O(n)$ as we are doing InOrder traversal two-times & $O(n \log n)$ for sorting the list items.

Space: $O(n)$ + recursion stack space. $O(n)$ for storing elements in the arraylist.

```

static void traversal(Node root, ArrayList<Integer> list){ 3 usages
    if (root==null) return;
    traversal(root.left,list);
    list.add(root.data);
    traversal(root.right,list);
}
static void helper(Node root, ArrayList<Integer> list,int[] index){
    if (root==null) return;
    helper(root.left,list,index);
    if (root.data != list.get(index[0])){
        root.data = list.get(index[0]);
    }
    index[0] = index[0]+1;
    helper(root.right,list,index);
}
static void bruteForce(Node root){ no usages
    ArrayList<Integer> inOrder = new ArrayList<>();
    traversal(root,inOrder);
    Collections.sort(inOrder);

    int[] index = new int[] {0};
    helper(root,inOrder,index);
    System.out.println(levelOrder(root));
}

```

Optimal:

A correct BST always gives a **sorted order** when we do an inorder traversal.

But when **two nodes are swapped**, this sorted order breaks.

Two Types of Invalid BST Cases:

1 Non-adjacent swap (two violations)

Example: 1, 4, 3, 2, 5

Violations:

4 > 3 → first = 4, middle = 3

3 > 2 → last = 2

Swap → first and last.

2 Adjacent swap (one violation)

Example:

1, 3, 2, 4, 5

Violation:

$3 > 2 \rightarrow \text{first} = 3, \text{middle} = 2$

Only one violation $\rightarrow$ swap first and middle.

Implementation Steps:

1. Create four pointers: prev, first, middle, last.
2. Do an inorder traversal. Keep checking if the current node breaks the sorted order.
3. Track nodes that are incorrectly placed:
 - First time $\rightarrow$ save first and middle
 - Second time $\rightarrow$ save last
4. After traversal:
 - If there are **two violations** $\rightarrow$ swap first and last
 - If only **one violation** $\rightarrow$ swap first and middle

This restores the BST.

Time: $O(N)$ where **N** = number of nodes in the tree as we are doing in-order traversal.

Space : recursion stack space

```

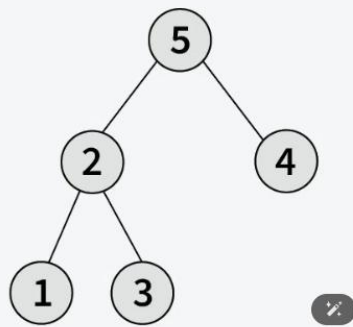
TreeNode prev = null, first = null, middle=null, last=null;
void helper(TreeNode root){
    if (root==null) return;
    helper(root.left);
    if (prev != null && prev.val>root.val){
        if (first == null) {
            first = prev;
            middle = root;
        }
        else{
            last = root;
        }
    }
    prev = root;
    helper(root.right);
}
public void recoverTree(TreeNode root) {
    helper(root);
    if (last==null){
        int temp = first.val;
        first.val = middle.val;
        middle.val = temp;
    }
    else{
        int temp = first.val;
        first.val = last.val;
        last.val = temp;
    }
}
}

```

You're given a binary tree. Your task is to find the size of the largest subtree within this binary tree that also satisfies the properties of a Binary Search Tree (BST). The size of a subtree is defined as the number of nodes it contains.

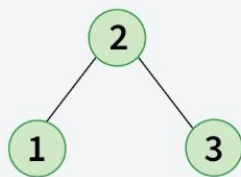
Note: A subtree of the binary tree is considered a BST if for every node in that subtree, the left child is less than the node, and the right child is greater than the node, without any duplicate values in the subtree.

Input: root = [5, 2, 4, 1, 3]



Output: 3

Explanation: The following sub-tree is a BST of size 3



Brute-force:

Using logic of validate a BST where we will pass address of each node to the validateBST() which will validate it & returns true if it's a bst otherwise, false. Whenever we get true will pass the address to helper() to get the number of nodes(size) in that particular sub-tree & store it inside some integer variable & whenever we get the number of nodes for any particular sub-tree will replace the integer variable with the current size of sub-tree. At the end will return integer variable value.

Time= $O(n)*O(n)$ $O(n)$ for traversing given tree & $O(n)$ for validating each sub-tree.

Space = recursion stack space.

Optimal-approach:

We are given a binary tree (NOT necessarily a BST).

We must find the size of the largest subtree that is a valid BST.

To do that for every node, we need to know:

1. Minimum value in its subtree
2. Maximum value in its subtree
3. Size of the largest BST in its subtree

So, for each node, we compute a Pair: **(min value, max value, size of largest BST)**

How do we know a subtree is a BST?

For a node, its subtree is BST if:

max value in left subtree < root.data < min value in right subtree

If this condition is true → current node is the root of a BST

Otherwise → not a BST

```
class Pair{
    int min,max,size;

    Pair(int min,int max,int size){
        this.min = min;
        this.max=max;
        this.size=size;
    }
}

class Solution {

    Pair optimal(Node root){
        if (root==null) return new Pair(Integer.MAX_VALUE,Integer.MIN_VALUE,0);
        Pair l = optimal(root.left);
        Pair r = optimal(root.right);

        if (l.max < root.data && r.min > root.data){
            int newMin = (root.left == null) ? root.data : l.min;
            int newMax = (root.right == null) ? root.data : r.max;
            return new Pair(newMin, newMax, l.size + r.size + 1);
        }
        return new Pair(Integer.MIN_VALUE,Integer.MAX_VALUE,Math.max(l.size,r.size));
    }

    int largestBst(Node root) {
        // Write your code here
        Pair res = optimal(root);
        return res.size;
    }
}
```

Time: $O(n)$ as we are doing single traversal.

Given a BST and a value k, the task is to delete the nodes having values greater than or equal to k.

Input:

```
4
 / \
1   9
k = 2
```

Output:

```
1
```

Optimal:

In a BST:

- All values in the left subtree are $< \text{root.data}$
- All values in the right subtree are $> \text{root.data}$

So, while deleting nodes:

Case 1 $\rightarrow$ If $\text{root.data} \geq k$

The entire **right subtree** is also $\geq k$, so none of those nodes should remain.

Thus we:

- Ignore the current root
- Ignore the right subtree
- Recursively process only the **left subtree**
- Return the result of left subtree (it replaces current node)

Case 2 $\rightarrow$ If $\text{root.data} < k$

The root is valid and must stay.

But its right subtree may still contain nodes $\geq k$, so we:

- Recursively process the right subtree
- Attach the processed subtree back to root.right
- Return the root as it is valid

This naturally removes all unwanted nodes while maintaining BST structure.

```
class Solution {
    Node helper(Node root, int k){
        if(root==null) return null;
        if(root.data>=k){
            Node curr = helper(root.left,k);
            return curr;
        }
        root.right = helper(root.right,k);
        return root;
    }

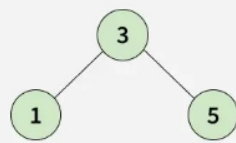
    public Node deleteNode(Node root, int k) {
        // add code here.
        return helper(root,k);
    }
}
```

Time = $O(n)$

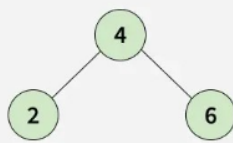
Space = recursion stack space

You are given the root of two BSTs, you have to merge this two BST and return the in-order traversal of the new BST.

Input: root1 = [3, 1, 5], root2 = [4, 2, 6]



First BST



Second BST

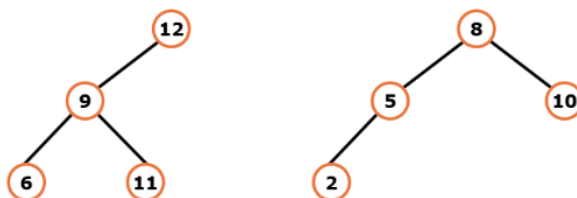
Output: [1, 2, 3, 4, 5, 6]

Explanation: After merging and sorting the two BST we get [1, 2, 3, 4, 5, 6].

Brute-force:

A naive approach to solve this problem is to simply traverse both of the trees using any technique and store all the elements in a list. Once we have all the elements, we can simply sort our list to get the elements of merged BST in sorted form.

- Traverse both BSTs using any traversal method (preorder, inorder, or postorder) and collect all elements in a single Array-list.
- Sort the list.
- Return the sorted list as the merged elements of both BSTs.



list =

6	9	11	12	2	5	8	10
---	---	----	----	---	---	---	----

Sorted list =

2	5	6	8	9	10	11	12
---	---	---	---	---	----	----	----

Time Complexity: $O((n+m) \cdot \log(n+m))$, we traverse both the BSTs and sort all the elements.

Space Complexity: $O(m+n)$, additional space required for storing elements of the two BSTs.

Optimal:

We know that inorder traversal of any BST returns the elements in sorted order. Thus, we will use this property to optimize our solution. We can simply traverse both the BSTs one by

one and store their elements in separate lists. Since, both these lists are individually sorted, we can just merge these sorted lists to get our elements of merged BST in sorted form.

- Traverse both BSTs using inorder traversal method and collect all elements in separate lists.
- Create an additional list to store the elements of the two sorted lists.
- Merge the sorted lists using a two pointer approach.
- Return the sorted list as the merged elements of both BSTs.

```
class Solution {
    void helper(Node root, ArrayList<Integer> res){
        if(root==null) return;
        helper(root.left, res);
        res.add(root.data);
        helper(root.right, res);
    }

    public ArrayList<Integer> merge(Node root1, Node root2) {
        // code here
        ArrayList<Integer> l1 = new ArrayList<>();
        ArrayList<Integer> l2 = new ArrayList<>();
        helper(root1, l1);
        helper(root2, l2);

        int i=0, j=0;
        ArrayList<Integer> res = new ArrayList<>(l1.size() + l2.size());
        while(i<l1.size() && j<l2.size()){
            if(l1.get(i)<l2.get(j)) res.add(l1.get(i++));
            else res.add(l2.get(j++));
        }
        while(i<l1.size()) res.add(l1.get(i++));
        while(j<l2.size()) res.add(l2.get(j++));

        return res;
    }
}
```

Time Complexity: $O(n+m)$, we traverse both the BSTs and merge two sorted lists.

Space Complexity: $O(m+n)$, additional space required for storing elements of the two BSTs.